

```
$ mysql -e "EXPLAIN SELECT * FROM users WHERE email = ?"
```



// $O(n)$ $O(\log n)$

B-Tree

□□□

EXPLAIN

□□□□

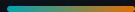
□□□□

□□□□□□

□□

□□□□□

\$ cat index-plan.md



01 /



vs



02 /



B-Tree



EXPLAIN

03 /



04 /



100 3000ms 3ms

PART 01



1000 000000 Apple

□□□□□□□□

```
1  □□□□
2  [1] → [2] → [3] → ... → [1000000]
3
4  □□□email = 'hua@email.com'
5  遍历 1 遍历
```

EXPLAIN

type: ALL□□□□□□

rows: 1000000

🕒 $O(n)$ - □□□□□□□□

□□□□□□□□

```
1  遍历      root
2           /      \
3         branch    branch
4           /      \
5        leaf      leaf → □□□□
```

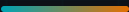
EXPLAIN

type: ref□□□□□□

rows: 1

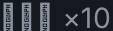
🕒 $O(\log n)$ - 3~4 次 I/O □□

□□□□□□□□□□



□□□□	□□□	□□□	□□□□
1,000	2ms	1ms	2x
10,000	25ms	1ms	25x
100,000	300ms	2ms	150x
1,000,000	3,500ms	3ms	1000x+

O(n) □□□□


 ×10
 
 ×10

O(log n) □□□□


 ×10
 

000<1000 000000000000>10 0000000000

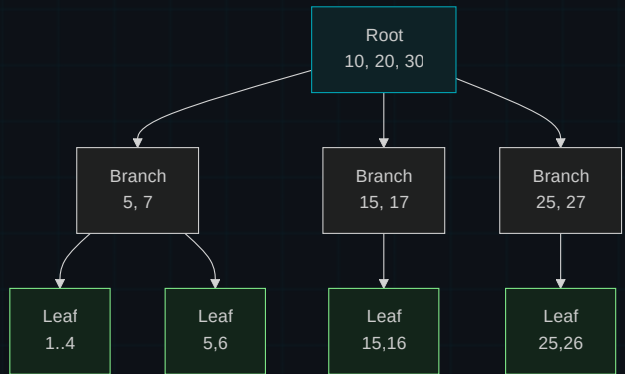
PART 02

B-Tree □□

□□□□ 3~4 □ I/O□

B-Tree

NO GRAPH



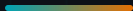
□□
□□□□□□□□

□□
□□□□□□□□

□□□□
□□□□□□□□

□□□
□□□□I/O □□

□□□□□□□□□□



```
1  -- 创建索引
2  CREATE INDEX idx_user_email ON users(email);
3
4  -- 查询
5  SELECT * FROM users WHERE email = 'hua@email.com';
6  -- type: ALL, rows: 1000000 → type: ref, rows: 1
```

□□□□□□□□



□□□□□

□□□□PRIMARY KEY □□□□□□□□□□□□

\$ 创建索引 EXPLAIN 类型 ref / range 扫描 ALL 扫描

PART 03



□□□□□

```
1  -- WHERE □□
2  CREATE INDEX idx_order_status
3      ON orders(status);
4
5  -- JOIN □□□
6  CREATE INDEX idx_order_user_id
7      ON orders(user_id);
8
9  -- ORDER BY □□
10 CREATE INDEX idx_order_created
11     ON orders(created_at);
```

□□□□□□

□□□□□□
gender □□□□□□□□□□□□

□□□□□□□
last_login_time □□□□□□□□□□

□□□□□□□
description □□□□□□□□□□□□



```
1  -- 100 rows
2  SELECT * FROM orders
3  WHERE user_id = 123 AND status = 'completed'
4         AND order_date >= '2024-01-01';
5
6  -- Create index
7  CREATE INDEX idx_orders_user_status_date
8         ON orders(user_id, status, order_date);
9  -- rows: 1000000 → 1200 8000
```

1. user_id

1000000

2. status

1200000

3. order_date

8000000

PART 04



0000 ≠ 00000

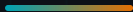
CHECKPOINT_01

name 姓 B-Tree 索引

A WHERE UPPER(name) = 'ZHANG'

B WHERE name LIKE '%姓'

C WHERE name LIKE '姓%'



```
1 SELECT * FROM users WHERE UPPER(name) = 'ZHANG'; -- ✗ 大小写敏感
2 SELECT * FROM users WHERE name = 'Zhang'; -- ✓ 不敏感
3
4 SELECT * FROM users WHERE name LIKE '%Z%'; -- ✗ 大小写敏感
5 SELECT * FROM users WHERE name LIKE '%z%'; -- ✓ 不敏感
6
7 SELECT * FROM users WHERE age + 10 > 35; -- ✗ 算术运算
8 SELECT * FROM users WHERE age > 25; -- ✓ 比较运算
```

SQL 语句	MySQL 8.0+ 支持	MySQL 8.0+ 不支持
SELECT * FROM users WHERE name = 'Zhang';	支持	不支持
SELECT * FROM users WHERE name LIKE '%Z%';	支持	不支持
SELECT * FROM users WHERE age + 10 > 35;	支持	不支持
SELECT * FROM users WHERE age > 25;	支持	不支持



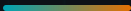
GAIN

- ✓ 時間 $O(n) \rightarrow O(\log n)$
- ✓ ORDER BY / GROUP BY 高速
- ✓ JOIN 高速

COST

- × 実行時間 10-30%増
- × INSERT / UPDATE / DELETE 高速
- × メモリ = 実行時間

80/20 法則 20% の実行時間削減を実現



B-Tree     $O(n)$  $O(\log n)$

```
type: ALL ☐ ☐ ☐ ☐ ref / range ☐ ☐ ☐
```

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □

□ □ □ □ □ □ □ □ □ □ □ □

□□□□□□□□ — □□□□□□□□□□

Query optimized.

Hash Aggregation – Hash Join

```
$ EXPLAIN SELECT ... -- type: ref, rows: 1  
b-tree → explain → composite → Hash
```